# HR KNOWLEDGE AGENT

Project Documentation & Technical Report

Submitted by

**Tejasvi Panoli**

AI and Gen AI Engineer Intern

*A conversational assistant for finding answers in HR policy documents*

# 1. Project Overview

The HR Knowledge Agent is a conversational application designed to help employees find information in company HR policy documents. Instead of manually searching lengthy files or asking HR about every routine question, users can type a question in everyday language and receive a response based on policy information made available to the application.

The project combines a web-based chat interface with a Python API and a language-model workflow. The frontend work includes an HR chat workspace, conversation and navigation concepts, a dashboard area, and a loading state while the assistant processes a question.

# 2. Problem Statement

HR policies can be spread across long documents and cover subjects such as leave, attendance, compensation, benefits, workplace conduct, remote work, IT usage, employee relations, and offboarding. Finding a relevant rule can be time-consuming when a user does not know the exact policy title or wording. The HR Knowledge Agent provides one conversational interface for asking questions about available policy content.

# 3. Project Objectives

- Make HR policy information easier to find through natural-language questions.
- Reduce the need to manually search long policy documents for routine information.
- Provide a straightforward conversational interface for employees.
- Use document-based retrieval as the intended basis for policy answers.
- Keep the frontend organized into reusable components and separate stylesheets.

# 4. Main Features

| Feature | Description |
|---|---|
| HR question-and-answer chat | Users can type questions about company policies in a chat interface. |
| Policy document processing | PDF text can be extracted for use in the document question-answering workflow. |
| Language-model response generation | The model generates a natural-language response using the question and context supplied by the application. |
| Conversation interface | Messages are displayed as user and assistant turns. |
| Chat navigation/history UI | The frontend contains chat-history and active-conversation concepts. |
| Loading feedback | A visible thinking/loading state indicates that the assistant is processing. |
| Dashboard and navigation | The interface is separated into reusable areas and components. |
| Separated styling | Global, chat, dashboard, sidebar, and app styles are kept in separate CSS files. |

# 5. Technology Stack and Tools

| Layer | Technology / Tool | Role |
|---|---|---|
| Frontend | React | Builds the interactive interface and reusable components. |
| Frontend tooling | Vite | Development server and build tooling for the React application. |
| Styling | CSS | Controls layout, colors, typography, responsiveness, and component appearance. |
| Backend API | Python + FastAPI | Provides HTTP endpoints and coordinates backend processing. |
| PDF processing | PyPDF2 | Extracts text from PDF documents. |
| LLM integration | LangChain / langchain-groq | Connects application logic to the language model. |
| Language model | openai/gpt-oss-120b via ChatGroq | AI Model. |
| Configuration | python-dotenv / .env | Loads configuration values such as API credentials from environment variables. |
| Architecture | RAG (Retrieval-Augmented Generation) | Retrieves relevant document context before the model generates an answer, where configured. |
| Vector search | FAISS | A vector similarity search library used in Retrieval-Augmented Generation (RAG) systems to store and retrieve relevant document embeddings. It can help the application find relevant HR policy information for employee queries. |
| Database | SQLite + SQLAlchemy | Stores user accounts, chat conversations, and chat messages through SQLAlchemy ORM models. |

# 6. Language Model

The application integrates a Large Language Model (LLM) through LangChain's ChatGroq integration to generate natural-language responses to employee queries. The model processes the user's question along with relevant HR policy information provided by the application and generates a contextual response.

This integration enables the HR Knowledge Agent to understand employee questions, interpret relevant policy information, and provide clear, conversational answers. It forms a core component of the application's question-answering functionality.

# 7. System Architecture

| Component | Responsibility | Input / Output |
|---|---|---|
| React frontend | Collects questions and displays messages and status. | Question entered by user; rendered assistant answer. |
| FastAPI backend | Receives requests and coordinates question/document processing. | API request; response payload. |
| PDF text extraction | Extracts readable text from policy PDFs. | PDF file; extracted text. |
| Retrieval layer (RAG) | Finds relevant passages from indexed policy content, where implemented. | Question and indexed content; relevant passages. |
| LLM integration | Creates a readable response using the question and supplied context. | Question/context; generated answer. |

# 8. Workflow Explained for Non-Technical Users

1. The employee opens the HR Knowledge Agent in a web browser.
2. The employee types a question, such as “What is the work-from-home policy?”
3. The React interface sends the question to the backend API.
4. The backend uses the available policy-document content to find information relevant to the question, where the retrieval pipeline is configured.
5. The language model receives the question and relevant context supplied by the application.
6. The model generates a response in ordinary language.
7. The backend returns the response to the frontend.
8. The chat interface displays the answer, and the employee can ask another question.

In simple terms, the application looks for policy information related to a question and uses an AI language model to explain that information conversationally.

# 9. Policy Documents and Knowledge Base

The handbook contains dummy/sample policy data intended to demonstrate how the HR Knowledge Agent processes and answers questions about HR policy documents.

- Workforce planning, recruitment, contracts, joining, and onboarding.
- Working hours, attendance, leave, holidays, compensation, payroll, and benefits.
- Performance management, learning, conduct, equal opportunity, and workplace harassment.
- Grievances, disciplinary procedures, health and safety, and flexible work.
- Information security, privacy, records, company property, and business travel.
- Resignation, termination, retirement, contractors, organisational changes, and offboarding.
- HR technology, AI-related decisions, policy governance, audit, compliance, and localisation review.

The policy content in the project handbook is dummy/sample data and is used for demonstration and testing.

# 10. Database and Data Model

The project uses SQLite as its relational database and SQLAlchemy as the Object-Relational Mapper (ORM). The database model defines three tables: users, chats, and messages. These tables store account details, conversation metadata, and the messages exchanged with the HR Knowledge Agent.

The database relationships connect each user to their chats and each chat to its messages. SQLAlchemy relationships provide convenient access between these related records.

| Data area | Confirmed information | Database status |
|---|---|---|
| users | id INTEGER PRIMARY KEY; name VARCHAR(100) NOT NULL; email VARCHAR(255) UNIQUE NOT NULL; hashed_password VARCHAR(255) NOT NULL; created_at DATETIME. | Stores user account details. Email is unique and indexed. One user can have multiple chats. |
| chats | id INTEGER PRIMARY KEY; title VARCHAR(255) DEFAULT 'New Chat'; user_id INTEGER NOT NULL FOREIGN KEY → users.id; created_at DATETIME; updated_at DATETIME. | Stores chat/conversation metadata. Each chat belongs to one user and can contain multiple messages. |
| messages | id INTEGER PRIMARY KEY; chat_id INTEGER NOT NULL FOREIGN KEY → chats.id; role VARCHAR(20) NOT NULL; | Stores each conversation message, its role, content, optional source information, and creation time. |

| | | |
|---|---|---|
| | content TEXT NOT NULL; sources TEXT NULL; created_at DATETIME. | |

The schema below reflects the SQLAlchemy model definitions used by the project.

Relationships and deletion behavior: User.chats links users to their chats; Chat.messages links chats to messages and orders messages by ID. Deleting a user cascades through their associated chats, and deleting a chat cascades through its associated messages. The user_id and chat_id foreign-key columns are indexed.

## 11. Frontend Component and Styling Structure

The frontend is organised into reusable React components. Styling has been separated to make the code easier to maintain and to distinguish global rules from component-specific rules.

| File / area | Purpose |
|---|---|
| App.jsx | Application-level composition and page/navigation rendering. |
| App.css | Application-level layout and styling. |
| styles/global.css | Global styles, shared defaults, and common design rules. |
| Chat component CSS (chat.css) | Chat workspace, messages, composer, and thinking/loading indicator styles. |
| Dashboard CSS (dashboard.css) | Dashboard-specific layout and component styles. |
| Sidebar CSS (sidebar.css) | Sidebar, navigation, chat history, and profile-related styles. |

Exact component filenames and import paths should match the final repository structure.

## 12. Backend and API Integration

FastAPI is used as the backend framework to handle communication between the frontend and the application's core functionalities. It provides API endpoints through which the frontend can send requests and receive responses.

The backend supports PDF document uploads and text extraction using PyPDF2. The extracted HR policy content is processed to provide relevant information for employee queries. The application integrates the ChatGroq language model to generate responses based on the available policy information.

The backend acts as the central layer connecting the frontend, document-processing functionality, AI model, and database. This structure helps organize the application's operations and enables communication between its different components.

## 13. Conclusion

The project combines a React and Vite frontend with a Python and FastAPI backend, SQLAlchemy-based SQLite persistence, PDF text extraction, and language-model integration through LangChain and ChatGroq. The RAG workflow is designed to ground responses in relevant policy-document content.